Report: Assignment 4 - Facade Pattern

Home Theater

by Abraimov Yeskendir, SE-2502

1. Introduction

For this assignment I chose Option B - Home Theater. I picked it because it is easy to imagine: a movie night needs a lot of devices, and turning them on one by one every time is annoying.

My subsystem has 6 devices: `Amplifier`, `DvdPlayer`, `Projector`, `Screen`, `TheaterLights` and `PopcornPopper`. To start one movie without a facade, the client has to make 13 calls, and to end it - 8 more. The order is important too, because the devices check each other:

- the amplifier and the projector don't accept a DVD player that is still off, so the DVD player has to be turned on first;
- volume, popcorn and widescreen mode work only after the device is on;
- if the projector lamp is burned out, the projector won't start, so everything that was already turned on has to be turned off again.

If every client did all of this by itself, every client would need to know all 6 classes and all these rules. So I made `HomeTheaterFacade` with two methods, `watchMovie(title)` and `endMovie()`.

`Main` works only with the facade. It gets it from `HomeTheaterInstaller`, which creates all the devices. Lecture 6 says that usually the client is just given a ready facade and doesn't build it by itself, so I did it this way. The devices are still public, and `AdvancedClient` uses them directly.

2. UML Class Diagram

At the top are the two clients, in the middle is the facade, at the bottom is the subsystem. All arrows go down into the subsystem and none go back up, because the devices don't know about the facade. The red dashed arrows are `AdvancedClient`, which goes around the facade.

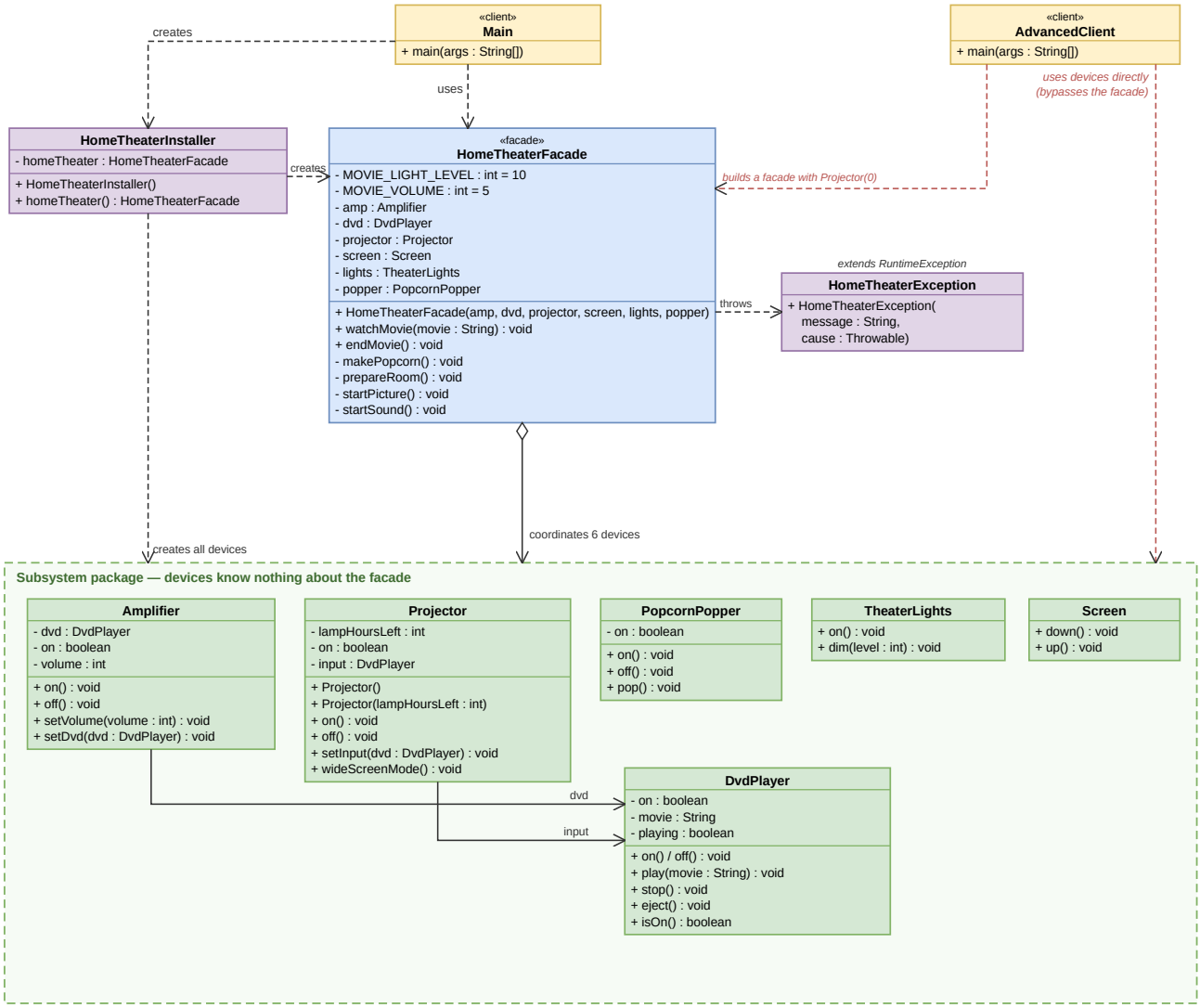


Figure 1 - HomeTheaterFacade in front of the home theater subsystem

Role	Class
Facade	`HomeTheaterFacade`
Complex Subsystem	`Amplifier`, `DvdPlayer`, `Projector`, `Screen`, `TheaterLights`, `PopcornPopper`
Client	`Main`
Helpers	`HomeTheaterInstaller` (creates the devices), `HomeTheaterException` (the facade's error)

3. Clean Code Principles Applied

For this section I used chapters 6, 7 and 8 of *Clean Code*: Objects and Data Structures, Error Handling and Boundaries. They fit Facade well, because the whole pattern is about what a class hides, what it shows, and what happens at the border between the client and the subsystem.

1. Objects Hide Their Data and Expose Behavior (Ch. 6)

Chapter 6 says: "Objects hide their data behind abstractions and expose functions that operate on that data" (p. 95), and "the worst option is to blithely add getters and setters". In my project all fields are `private` -

both in the devices and in the facade. The facade doesn't have getters like `getAmplifier()`. It shows only actions: `watchMovie()` and `endMovie()`. The devices also don't have a getter for every field. The only one is `isOn()` in `DvdPlayer`, because the amplifier and the projector really need it to check the player.

Before - the facade shows its data:

```
public class HomeTheaterFacade {
    public Amplifier amp;
    public DvdPlayer dvd;
    ...
    public Amplifier getAmplifier() { return amp; }
    public Projector getProjector() { return projector; }
}
```

After - HomeTheaterFacade.java:

```
private final Amplifier amp;
private final DvdPlayer dvd;
private final Projector projector;
private final Screen screen;
private final TheaterLights lights;
private final PopcornPopper popper;
...
public void watchMovie(String movie){
...
public void endMovie() {
```

`final` also means the devices can't be replaced after the facade is created.

2. The Law of Demeter - Talk to Friends, Not to Strangers (Ch. 6)

The Law of Demeter (p. 97-98) says that a method should only call methods of its own class, of objects it creates, of its arguments, and of objects held in its fields - "talk to friends, not to strangers". Lecture 6 calls this the Principle of Least Knowledge. The facade follows it: `watchMovie()` only calls the devices stored in its own fields. The client follows it too: `Main` only talks to the facade. It never reaches through the facade to a device, so there are no "train wrecks" like `homeTheater.getAmplifier().setVolume(7)`.

The book also says: "If ctxt is an object, we should be telling it to do something; we should not be asking it about its internals" (p. 99). That is exactly how the client uses the facade - it tells it "watch a movie", not "give me the amplifier".

Before - the client navigates through the facade:

```
homeTheater.getPopper().on();
homeTheater.getLights().dim(10);
homeTheater.getAmplifier().setVolume(5);
...
```

After - Main.java:

```
HomeTheaterInstaller installer = new HomeTheaterInstaller();
HomeTheaterFacade homeTheater = installer.homeTheater();

homeTheater.watchMovie("Interstellar");
System.out.println();
homeTheater.endMovie();
```

3. Use Exceptions Rather Than Return Codes (Ch. 7)

Chapter 7 says that with error codes "the caller must check for errors immediately after the call. Unfortunately, it's easy to forget. For this reason it is better to throw an exception" (p. 104). My devices

throw an exception when they are used wrong: `IllegalStateException` if the device is off, and `IllegalArgumentException` if the value is wrong. If the facade calls something in the wrong order, it finds out right away instead of getting a silent bug.

All my exceptions are unchecked (`RuntimeException`). The book explains that checked exceptions break the Open/Closed Principle: if a low-level method starts throwing one, every method above it has to change its signature (p. 106-107). With unchecked exceptions `watchMovie()` doesn't need a `throws` list.

Before - error code that the caller can forget to check:

```
public boolean setVolume(int volume) {
    if (!on) {
        return false;
    }
    this.volume = volume;
    return true;
}

// caller
amp.setVolume(5);   // returned false, nobody noticed
```

After - Amplifier.java:

```
public void setVolume(int volume) {
    if (!on) {
        throw new IllegalStateException("Amplifier is off");
    }
    if (volume < 0 || volume > 11) {
        throw new IllegalArgumentException("Volume must be from 0 to 11, got " + volume);
    }
    this.volume = volume;
    System.out.println("Amplifier volume set to " + volume);
}
```

4. Try Blocks Are Like Transactions, With Context and One Exception Type (Ch. 7)

This is the most important principle for my facade. The book says: "try blocks are like transactions. Your catch has to leave your program in a consistent state, no matter what happens in the try" (p. 105). The facade calls many devices in a row, so something can fail in the middle. For example, with a burned out projector lamp the popcorn, the lights, the screen and the DVD player are already on. So my `catch` calls `endMovie()` and the room goes back to normal.

Two more rules from the same chapter are used here:

- **Provide context with exceptions** - "Mention the operation that failed and the type of failure" (p. 107). My message says both: *Could not start "Dune"* (the operation) and *Projector lamp is burned out* (the failure).
- **Define exception classes in terms of a caller's needs** - "Often a single exception class is fine for a particular area of code" (p. 109). The book wraps a library's three exceptions into one `PortDeviceFailure`. I did the same: device errors are wrapped into one `HomeTheaterException`, so the client catches one type and doesn't need to know what the devices throw inside. The original error is kept as the `cause`.

Before - the room stays half turned on:

```
public void watchMovie(String movie){
    popper.on();
    ...
    projector.on();      // throws, but popper, lights and screen stay on
    ...
}
```

After - HomeTheaterFacade.java:

```
try {
    makePopcorn();
    prepareRoom();
    startPicture();
    startSound();
    dvd.play(movie);
} catch (IllegalStateException e){
    //some device failed in the middle, so turn everything back off
    System.out.println("Something went wrong, turning everything off...");
    endMovie();
    throw new HomeTheaterException("Could not start \"" + movie + "\": " + e.getMessage(),
e);
}
```

For this to work, `endMovie()` itself must not fail, so `off()`, `stop()` and `eject()` just `return` if there is nothing to do. `AdvancedClient` shows it with `new Projector(0)`:

```
DVD Player on
Something went wrong, turning everything off...
Shutting movie theater down...
Popcorn Popper off
Theater Lights on
Screen going up
DVD Player off
Error: Could not start "Dune": Projector lamp is burned out
```

Wrong input is checked even before the `try`: an empty movie title is rejected before any device is turned on.

5. Clean Boundaries (Ch. 8)

Chapter 8 is about code we don't control, and it starts by saying that this can also be "components or subsystems" made by other teams (p. 113). For the client, the device classes are exactly that kind of subsystem. The book's main advice is: "We manage third-party boundaries by having very few places in the code that refer to them" (p. 120), and "It's better to depend on something you control than on something you don't control" (p. 120).

In the book, `Map` is hidden inside a small `Sensors` class, and this class is "tailored and constrained to meet the needs of the application. It results in code that is easier to understand and harder to misuse" (p. 115). My facade does the same with the subsystem: it shows only the two things the client needs. In my project only two places refer to the devices: `HomeTheaterFacade`, which uses them, and `HomeTheaterInstaller`, which creates them. `Main` has no import from the `Subsystem` package at all. If a device changes, only those two classes change.

Before (Lecture 6, HomeTheaterTestDrive) - the client creates and passes around every device:

```
public static void main(String[] args) {
    // Instantiate all components here
    HomeTheaterFacade homeTheater =
        new HomeTheaterFacade(amp, tuner, dvd, cd, projector, screen, lights, popper);
    homeTheater.watchMovie("Raiders of the Lost Ark");
}
```

After - HomeTheaterInstaller.java (the only place that creates devices):

```
public HomeTheaterInstaller(){
    homeTheater = new HomeTheaterFacade(new Amplifier(), new DvdPlayer(), new Projector(),
            new Screen(), new TheaterLights(), new PopcornPopper());
}
```

Main.java imports only the Facade package:

```
import Facade.HomeTheaterFacade;
import Facade.HomeTheaterInstaller;
```

4. Conclusion

What the facade hides. The client doesn't need to know that there are 6 device classes or that a movie needs 21 calls. It doesn't need to know that the DVD player must be on before the amplifier or the projector can use it, which volume and light level to use for a movie, or what to do if a device breaks. It just calls `watchMovie()` and `endMovie()`. If one device is replaced, only the facade and the installer change, and the client stays the same.

What stays directly accessible. Everything. The lecture calls this the "encapsulation myth" - a facade doesn't encapsulate the subsystem, it just gives an easier way to use it. All devices and their methods are still public. `AdvancedClient` turns the volume up to 11, which the facade never does. And the devices don't have any reference to the facade, so they work without it.

Trade-offs. The facade decides some things for the user - volume 5 and lights at 10%. For a normal movie night that is fine, but if you want something else you have to use the devices directly. The facade is also one more layer to go through when debugging. And it could grow into a god object, which the lecture lists as the main disadvantage. My subsystem has only movie devices, so one facade is enough. But if I added a radio, I would put it into a separate additional facade instead of adding it to `HomeTheaterFacade`.

One more thing I noticed: in the lecture example `amp.setDvd(dvd)` is called before `dvd.on()`. In my subsystem this throws an exception, because the amplifier doesn't take a DVD player that is off. So the facade is not just a list of calls - it really has to know the right order.

Compared to Adapter and Bridge. Adapter (Lecture 3) changes an interface so it fits what the client expects, and Facade makes an interface simpler. The difference is not how many classes they wrap, but why. Bridge from Assignment 3 was planned from the start to split the design into two hierarchies. Facade doesn't add any new hierarchy, it just puts one simple entry point in front of classes that already exist.

5. GitHub Repository

Repository: https://github.com/Naitskaut/SDP
Assignment 4 folder: https://github.com/Naitskaut/SDP/tree/main/Assignment%204